

PROJECT SYNOPSIS

MerchantIQ: An Evidence-Grounded ML and LLM Decision Intelligence System

Applied Machine Learning • Retrieval-Augmented Generation • LLM Inference • ML Systems Engineering

1. Executive Summary

MerchantIQ is an end-to-end AI decision-intelligence system that transforms merchant transaction data into evidence-grounded insights and actionable recommendations. The project combines data pipelines, analytics, machine learning, anomaly detection, forecasting, Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), structured outputs, API-based serving, evaluation and performance measurement.

Rather than functioning as a conventional dashboard or generic chatbot, MerchantIQ follows the workflow: **Data Ingestion** → **Analytics** → **ML Prediction** → **Evidence Retrieval** → **LLM Reasoning** → **Structured Recommendation** → **Evaluation** → **Monitoring**. The system is designed to demonstrate both business intelligence value and practical Applied ML/LLM engineering concepts.

2. Problem Statement

Merchants generate substantial transaction, payment and customer data, yet dashboards generally describe what happened without reliably explaining why it happened, what may happen next, or which action should be prioritized. Generic LLM systems may produce plausible but unsupported explanations. MerchantIQ addresses this gap by combining quantitative analytics and ML predictions with evidence-grounded LLM reasoning and measurable system performance.

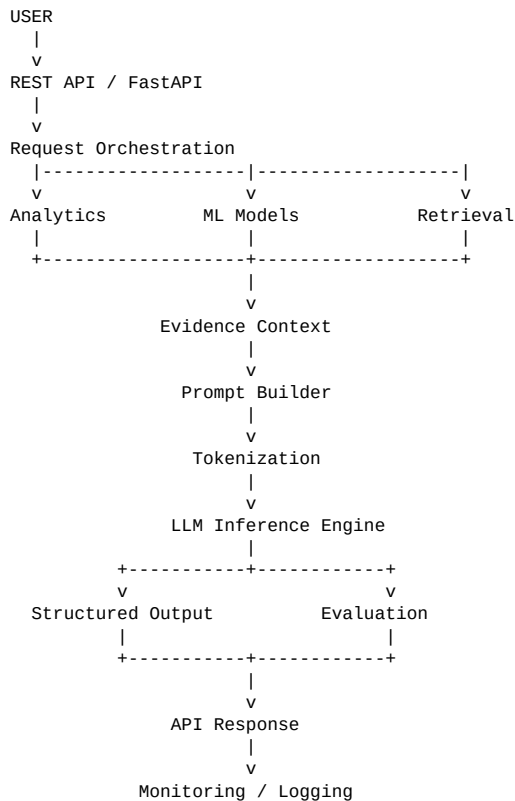
3. Objectives

- Build an end-to-end merchant analytics and decision-intelligence prototype.
- Apply ML for anomaly detection and short-term forecasting with measurable validation metrics.
- Implement embedding-based retrieval and evidence-grounded RAG.
- Generate structured explanations and next-best-action recommendations.
- Evaluate retrieval quality, groundedness, unsupported claims and output validity.
- Measure inference characteristics including latency, throughput and response performance.
- Expose the system through FastAPI and containerize it using Docker.

4. Core Project Capabilities

- **Data & Analytics:** KPI monitoring, revenue trends, transaction volume, payment success/failure, refunds and customer activity.
- **Machine Learning:** anomaly detection, forecasting and optional risk prediction.
- **Explainability:** feature importance and structured evidence generation.
- **RAG:** document preparation, chunking, embeddings, vector search and context assembly.
- **LLM Layer:** evidence-grounded reasoning, root-cause explanations and structured JSON outputs.
- **Decision Layer:** prioritized recommendations based on evidence, confidence and estimated impact.
- **Engineering:** FastAPI, Docker, logging, testing and performance experiments.

5. High-Level System Architecture



6. Machine Learning Methodology

Anomaly Detection: Isolation Forest and statistical/rolling-baseline approaches will be compared using precision, recall, F1 and false-positive rate where labelled data is available.

Forecasting: Naïve or seasonal baselines will be compared with an appropriate ML/time-series model using MAE, RMSE and MAPE where suitable.

Explainability: Feature importance or SHAP-style explanations will be converted into structured evidence for downstream reasoning.

7. Retrieval-Augmented Generation Pipeline

The RAG layer ensures that the LLM reasons over merchant-specific evidence instead of relying solely on parametric knowledge.

OFFLINE:
Merchant Data → Analytics/ML Outputs → Evidence Documents
→ Chunking → Embeddings → Vector Index

ONLINE:
User Query → Query Embedding → Similarity Search → Top-K Evidence
→ Context Assembly → LLM Inference → Structured Response

8. LLM and Inference Concepts

The project will document and practically investigate the request lifecycle: **Prompt** → **Tokenization** → **Embeddings** → **Transformer Processing** → **Attention** → **Autoregressive Decoding** → **Output Validation** → **API Response**. Key concepts include context length, prefill versus decode, latency, throughput, batching and KV-cache concepts. Advanced frameworks such as vLLM may be evaluated where computational resources permit, but low-level CUDA implementation is outside the MVP scope.

9. PyTorch and Transformer Component

- Tensors and tensor operations

- Autograd and neural network modules
- Dataset/DataLoader and training loops
- CPU/GPU execution and inference mode
- Mixed precision where applicable
- Token embeddings, positional information, self-attention, multi-head attention and causal masking

10. API, Deployment and Monitoring

FastAPI will provide validated REST endpoints for analysis, natural-language queries, recommendations, health checks and metrics. Docker will provide reproducible deployment. Logging and lightweight monitoring will capture request failures, latency and selected performance indicators.

11. Evaluation Framework

- **ML:** MAE, RMSE, MAPE, precision, recall, F1 and false-positive rate.
- **Retrieval:** Precision@K, Recall@K, context relevance and relevant evidence coverage.
- **LLM:** groundedness, unsupported-claim rate, evidence coverage and structured-output validity.
- **System:** end-to-end latency, retrieval latency, generation latency, throughput, error rate and failure handling.
- **Business:** recommendation acceptance and measurable outcome improvement where simulation/data permits.

12. Experimental Component

The project will include at least one serious hands-on systems experiment rather than claiming unverified infrastructure expertise. Example research questions include:

- How does context length affect end-to-end LLM latency?
- How does batch size affect throughput?
- How does retrieval Top-K affect groundedness and answer quality?
- Where feasible, how does quantization affect latency, memory use and response quality?

13. Project Implementation Structure

```

MerchantIQ/
├── data/
│   ├── raw/
│   ├── processed/
│   └── synthetic/
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_feature_engineering.ipynb
│   ├── 03_anomaly_detection.ipynb
│   ├── 04_forecasting.ipynb
│   └── 05_llm_evaluation.ipynb
├── src/
│   ├── data/           (ingestion, preprocessing, validation)
│   ├── analytics/      (KPI engine, trends, root-cause analysis)
│   ├── ml/             (anomaly detection, forecasting, evaluation)
│   ├── retrieval/      (documents, embeddings, vector store, retriever)
│   ├── llm/            (prompting, inference, structured output)
│   ├── agents/         (decision and recommendation logic)
│   ├── evaluation/     (ML, retrieval, LLM, performance metrics)
│   └── monitoring/     (logging and metrics)
├── api/
│   ├── main.py
│   ├── routes/
│   └── schemas/
├── frontend/
│   └── streamlit_app.py
├── tests/
│   ├── test_ml.py
│   ├── test_retrieval.py
│   ├── test_llm.py
│   └── test_api.py
└── experiments/

```

```
■ ■■■ context_length_vs_latency.py
■ ■■■ batch_size_vs_throughput.py
■ ■■■ retrieval_topk_evaluation.py
■■■ Dockerfile
■■■ docker-compose.yml
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
```

14. Technology Stack

Data: Python, Pandas, NumPy, SQL/MySQL/PostgreSQL

ML: scikit-learn, PyTorch

LLM/RAG: Hugging Face Transformers, Sentence Transformers, FAISS/Chroma

API: FastAPI, Pydantic

Deployment: Docker, Linux

Visualization: Streamlit, Plotly/Matplotlib

Engineering: Git/GitHub, testing, logging and experiment tracking where appropriate.

15. Scope Control

The MVP focuses on five connected capabilities: (1) merchant performance diagnosis, (2) anomaly detection and forecasting, (3) embedding-based evidence retrieval, (4) evidence-grounded LLM reasoning, and (5) measurable API/inference performance. CUDA kernels, Triton, distributed multi-GPU serving, NCCL and Kubernetes are future extensions rather than claimed MVP capabilities.

16. Deliverables

- Working end-to-end MerchantIQ prototype
- Data ingestion and analytics pipeline
- Anomaly detection and forecasting models with evaluation
- Embedding and RAG pipeline
- Evidence-grounded LLM reasoning module
- Structured output validation
- FastAPI service and Streamlit interface
- Docker configuration
- Performance benchmarking experiments
- LLM/RAG evaluation test cases
- Architecture documentation and clean GitHub repository

17. Expected Novelty and Outcome

MerchantIQ is not positioned as a generic 'chat with your data' application. Its distinctive architecture connects quantitative analytics and ML predictions with evidence retrieval and controlled LLM reasoning. The system treats analytics as the source of facts, retrieval as the mechanism for selecting relevant evidence, and the LLM as the component that synthesizes and communicates grounded conclusions.

Success will therefore be measured not only by fluent responses but by predictive performance, retrieval quality, groundedness, structured-output reliability, latency, throughput, failure handling and reproducibility. The final project serves as a practical portfolio demonstrating Applied Machine Learning, RAG, LLM applications and ML systems engineering.